

Pythonのパフォーマンス向上のための 多言語トランスコンパイラの実装

北原玲

情報デザイン・コミュニケーション工学コース 数理波動システム研究室

1 背景・目的

Python はその可読性と保守性の高さから広く利用されているが、インタプリタ言語であるため実行速度が遅い。この制約を克服するため、ソフトウェア開発などでは高速な C 言語や Fortran で記述されたライブラリを併用することが一般的である。しかし、異なる言語間での開発は効率が悪く、保守性を損なう可能性がある。この課題を解決する手段として、単一の言語から複数の言語への翻訳が行われている。また、大規模な開発では、Python のようなインタプリタ言語の実行速度を向上させるために、コード生成時に最適化処理を行う必要がある。これらを実現するには、トランスコンパイラ [1] を使用するのが現実的である。

トランスコンパイラとは、あるプログラミング言語を同水準の言語に変換するソフトウェアの総称である。例としては、Cython や Typescript、Haxe[3] などが挙げられる。本研究では Aqualis[2] という多言語トランスコンパイラを使用する。Aqualis は、保守性・可読性の高い Aqualis のソースコードから C 言語、Fortran、HTML、LaTeX、SVG(図面・グラフ) のソースファイルへの変換が可能となっている。また、C 言語及び Fortran に変換する際にはコードの最適化が行われ、プログラムを高速に実行することが可能である。しかし、現在の Aqualis には Python への変換機能は存在しない。

本研究の目的は、Aqualis に Python のソースコード生成機能を追加することと、Aqualis を用いて最適化処理を行った Python のコードを生成することである。

2 Aqualis

Aqualis は、F# というプログラミング言語のライブラリによって実装されている。そのため、その命令文は全て F# のメソッドとなっている。メソッドの引数としてラムダ式を渡すことで、疑似的な言語として機能している。例えば、変数宣言はソースコード 1 のように記述する。

ソースコード 1 Aqualis の変数宣言

```
1 ch.i <| fun x ->
```

```
2     x <== 1
3     print.c x
```

これは整数型の変数 x を宣言したときの例である。1 行目の “fun x ->” 以降の式がラムダ式となっており、変数 x を用いた処理が記述されている。“ch.i” は変数の生成と使用に関するクラス “ch” に属する整数型変数の宣言メソッドである。他に、実数型 (倍精度浮動小数点)、複素数型及びそれらの配列を宣言するメソッドが実装されている。上記のコードでは “x <== 1” で x に 1 を代入し、“print.c x” で変数 x を画面出力している。 x の使用範囲はインデントで指定できる。上記のコードでは 2 行目から 3 行目が x の使用可能な範囲である。この範囲外で変数のメモリ領域は他の変数に割り当てられ、プログラムのメモリ消費を抑えることに寄与している。

3 Python への変換機能の実装

Aqualis に Python への変換機能を実装する際は、既実装されている C 言語や Fortran の生成コードを参考に行った。しかし、関数名や記号を置き換えただけでは実装できなかった機能も存在する。その一つである多重ループからの脱出について述べる。

反復処理のコードはクラス `iter` のメソッドを用いて生成される。多重ループからの脱出は、`iter` の脱出機能を持つメソッドで自動的に生成される脱出関数を用いて行われる。この脱出関数はどのような位置からでも指定されたループを脱出することが求められる。例えば、

ソースコード 2 Aqualis の二重ループ

```
1 iter.loop <| fun (ex,i) ->
2     iter.loop <| fun (ey,j) ->
3         br.if1 (j.>=5) <| fun () ->
4             ex()
5         print.cc i j
```

ソースコード 2 のような無限ループを生成するメソッド `loop` を二つ用いた二重ループでは、外ループの脱出関数 `ex()` を用いたときは外ループと内ループを、内ループの脱出関数 `ey()` を用いたときは内ループのみを脱出しなければならない。

この機能を C 言語や Fortran では goto 文によって実装している。しかし、Python には goto 文がないため、if 文と break 文を用いて脱出関数を実装する必要があった。そのため、脱出関数内では脱出のためのフラグ変数と break 文を生成し、内ループの外に連続脱出用の条件文を生成するように iter のメソッドを実装した。実際にソースコード 2 から生成された Python のコードはソースコード 3 のようになる。

ソースコード 3 Aqualis で生成した Python コード

```

1 ic0001 = 1
2 while True:
3     ic0002 = 1
4     while True:
5         if ic0002 > 5:
6             flag = 10
7             break
8         print("%8d%8d" %(ic0001,ic0002))
9         flag = 11
10        ic0002 = ic0002+1
11    if flag < 11:
12        break
13    flag = 10
14    ic0001 = ic0001+1

```

上記のコードでは、無限ループ “while True:” が二重に生成されている。ループの末尾ではそれぞれのカウンタ変数と脱出用のフラグ変数 “flag” の値の更新が行われている。また、内ループのすぐ外にはフラグ変数を利用した連続脱出用の条件文が生成されている。脱出関数 ex() は、7 行目の “flag = 10” と 8 行目の “break” を出力した。連続脱出用の条件文は “flag < 11” であるため、5 行目の条件文を満たせば内ループから外ループの外まで脱出できる。

多重ループからの脱出のほかにもクラスを要素に持つ配列の生成や整数同士の剰余無視の除算演算子、参照渡し関数の生成などの実装にも独自の工夫を必要とした。

4 実行時間の比較

Aqualis は最適化処理の一つとして、関数は基本的にインライン展開されるようになっている。これによって、関数呼び出しに伴うオーバーヘッドを削減することができる。この削減が処理の効率化にどの程度貢献するのか、実行時間を比較するために Aqualis で生成されたコードと手書きのコードに対し、それぞれ二種類のテストプログラムを用意した。テスト A は、関数 h を 1000 回呼び出すプログラムである。テスト B は、関数 h を引数に受

け取る高階関数 f を 1000 回呼び出すプログラムとなっている。

表は、テスト A、B を 5 回実行したときの平均の計算時間である。プログラムは OS:Windows11、PC:WindowsPC(CPU:Intel(R) Core(TM) i5-9400)、環境:Windows Subsystem for Linux で実行した。

表 1 手書きと Aqualis の生成コードの実行時間

計測対象	生成方法	実行時間 [μs]
テスト A	手書き	136.2
テスト A	Aqualis	82.6
テスト B	手書き	206.6
テスト B	Aqualis	73.3

手書きのコードと Aqualis によって生成されたコードを比較した結果、関数 h の場合、高階関数 f の場合共に実行時間が短縮された。また、関数 h では手書きのコードの実行時間の約 60%、高階関数 f においては約 35% 程度の実行時間まで短縮することができた。高階関数 f の方が、関数 h の実行時間よりも相対的に短縮できた理由としては、高階関数 f が関数 h を呼び出す際に生じるオーバーヘッド分だけ手書きのコードの実行時間が増加した事が考えられる。このことから Aqualis の最適化処理によって Python の実行時間の短縮が可能であることが分かった。これにより、本研究の目的は達成された。

5 まとめ

本研究では、Aqualis に Python のソースコード生成機能を実装し、Aqualis を用いて最適化処理による Python の実行時間の短縮を図った。Aqualis に Python への変換機能を実装したことにより、他言語との連携が従来よりも簡単になった。Aqualis の生成コードと手書きのコードの比較では、実行時間の短縮に成功した。

参考文献

- [1] E. Ilyushin and D. Namiot, “On source-to-source compilers,” International Journal of Open Information Technologies, vol.4, no.5, pp.48–51, 2016.
- [2] 杉坂純一郎, “Sugisaka/Aqualis”, <https://github.com/Sugisaka/Aqualis>.
- [3] Nicolas Cannasse, “Haxe Foundation”, <https://github.com/HaxeFoundation>.